

1. **Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.**

Code:

```
import pandas as pd

# Create the dataset
data = {
    "CGPA": ["High", "High", "High", "Low", "Low", "High", "Low", "High", "Low", "High"],
    "Aptitude":
["High", "High", "High", "Low", "Low", "High", "Low", "High", "Low", "High"],
    "Communication":
["High", "High", "Medium", "Low", "Low", "High", "Medium", "High", "Low", "High"],
    "Placement": ["Yes", "Yes", "Yes", "No", "No", "Yes", "No", "Yes", "No", "Yes"]
}

df = pd.DataFrame(data)

print("Training Dataset:\n")
print(df)

# Extract positive examples
concepts = df.iloc[:, :-1].values
target = df.iloc[:, -1].values

# FIND-S Algorithm
hypothesis = None

for i in range(len(target)):
    if target[i] == "Yes":
        if hypothesis is None:
            hypothesis = concepts[i].copy()
        else:
            for j in range(len(hypothesis)):
                if hypothesis[j] != concepts[i][j]:
                    hypothesis[j] = "?"

print("\nMost Specific Hypothesis:")
print(hypothesis)
```

Output:

... Training Dataset:

	CGPA	Aptitude	Communication	Placement
0	High	High	High	Yes
1	High	High	High	Yes
2	High	High	Medium	Yes
3	Low	Low	Low	No
4	Low	Low	Low	No
5	High	High	High	Yes
6	Low	Low	Medium	No
7	High	High	High	Yes
8	Low	Low	Low	No
9	High	High	High	Yes

Most Specific Hypothesis:
['High' 'High' '?']

2. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples

Code:

```
import pandas as pd
import numpy as np
```

```
# Training Dataset
```

```
data = {
    "CGPA": ["High","High","High","Low","Low","High","Low","High","Low","High"],
    "Aptitude":
["High","High","High","Low","Low","High","Low","High","Low","High"],
    "Communication":
["High","High","Medium","Low","Low","High","Medium","High","Low","High"],
    "Placement": ["Yes","Yes","Yes","No","No","Yes","No","Yes","No","Yes"]
}
```

```
df = pd.DataFrame(data)
```

```
print("Training Dataset\n")
print(df)
```

```
concepts = np.array(df.iloc[:, :-1])
target = np.array(df.iloc[:, -1])
```

```

# Candidate Elimination Algorithm
def candidate_elimination(concepts, target):

    specific = concepts[0].copy()
    general = [["?" for i in range(len(specific))
                for j in range(len(specific))]

    print("\nInitial Specific Hypothesis:")
    print(specific)

    print("\nInitial General Hypothesis:")
    print(np.array(general))

    for i, h in enumerate(concepts):

        if target[i] == "Yes":

            for x in range(len(specific)):
                if h[x] != specific[x]:
                    specific[x] = "?"
                    general[x][x] = "?"

        elif target[i] == "No":

            for x in range(len(specific)):
                if h[x] != specific[x]:
                    general[x][x] = specific[x]
                else:
                    general[x][x] = "?"

    general = [g for g in general if g != ["?", "?", "?"]]

    return specific, general

specific, general = candidate_elimination(concepts, target)

print("\nFinal Specific Hypothesis:")
print(specific)

print("\nFinal General Hypothesis:")
for g in general:
    print(g)

```

Output:

		CGPA	Aptitude	Communication	Placement
***	0	High	High	High	Yes
	1	High	High	High	Yes
	2	High	High	Medium	Yes
	3	Low	Low	Low	No
	4	Low	Low	Low	No
	5	High	High	High	Yes
	6	Low	Low	Medium	No
	7	High	High	High	Yes
	8	Low	Low	Low	No
	9	High	High	High	Yes

Initial Specific Hypothesis:
['High' 'High' 'High']

Initial General Hypothesis:
[['?' '?' '?']
['?' '?' '?']
['?' '?' '?']]

Final Specific Hypothesis:
['High' 'High' '?']

Final General Hypothesis:
['High', '?', '?']
['?', 'High', '?']

3. **Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.**

Code:

```
# Decision Tree using ID3 Algorithm

import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# -----
# Dataset: Student Placement
# -----
```

```

data = {
    "CGPA": ["High","High","High","Low","Low","High","Low","High","Low","High"],
    "Aptitude":
["High","High","High","Low","Low","High","Low","High","Low","High"],
    "Communication":
["High","High","Medium","Low","Low","High","Medium","High","Low","High"],
    "Placement": ["Yes","Yes","Yes","No","No","Yes","No","Yes","No","Yes"]
}

```

```

df = pd.DataFrame(data)

```

```

print("Training Dataset\n")
print(df)

```

```

# -----
# Convert Categorical Data into Numbers
# -----

```

```

encoder = LabelEncoder()

```

```

df["CGPA"] = encoder.fit_transform(df["CGPA"])
df["Aptitude"] = encoder.fit_transform(df["Aptitude"])
df["Communication"] = encoder.fit_transform(df["Communication"])
df["Placement"] = encoder.fit_transform(df["Placement"])

```

```

# Features and Target

```

```

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

```

```

# -----
# Create Decision Tree using ID3
# -----

```

```

model = DecisionTreeClassifier(criterion="entropy", random_state=42)

```

```

# Train the Model

```

```

model.fit(X, y)

```

```

# -----
# Test Sample
# High = 0
# Low = 1
# Communication:

```

```

# High = 0
# Medium = 1
# Low = 2
# -----

sample = pd.DataFrame({
    "CGPA": [0],
    "Aptitude": [0],
    "Communication": [0]
})

# Prediction
prediction = model.predict(sample)

if prediction[0] == 1:
    print("\nPrediction : Yes (Placed)")
else:
    print("\nPrediction : No (Not Placed)")

```

Output:

... Training Dataset

	CGPA	Aptitude	Communication	Placement
0	High	High	High	Yes
1	High	High	High	Yes
2	High	High	Medium	Yes
3	Low	Low	Low	No
4	Low	Low	Low	No
5	High	High	High	Yes
6	Low	Low	Medium	No
7	High	High	High	Yes
8	Low	Low	Low	No
9	High	High	High	Yes

Prediction : Yes (Placed)

4. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Code:

```
# Artificial Neural Network using Backpropagation

import pandas as pd
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import LabelEncoder

# Dataset

data = {
    "CGPA": ["High", "High", "High", "Low", "Low", "High", "Low", "High", "Low", "High"],
    "Aptitude":
["High", "High", "High", "Low", "Low", "High", "Low", "High", "Low", "High"],
    "Communication":
["High", "High", "Medium", "Low", "Low", "High", "Medium", "High", "Low", "High"],
    "Placement": ["Yes", "Yes", "Yes", "No", "No", "Yes", "No", "Yes", "No", "Yes"]
}

df = pd.DataFrame(data)

print("Training Dataset\n")
print(df)

encoder = LabelEncoder()

df["CGPA"] = encoder.fit_transform(df["CGPA"])
df["Aptitude"] = encoder.fit_transform(df["Aptitude"])
df["Communication"] = encoder.fit_transform(df["Communication"])
df["Placement"] = encoder.fit_transform(df["Placement"])

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

ann = MLPClassifier(
    hidden_layer_sizes=(5,),
    activation='logistic',
    solver='adam',
    learning_rate_init=0.01,
    max_iter=3000,
```

```

    random_state=42
)

ann.fit(X, y)

# Test Sample
sample = pd.DataFrame({
    "CGPA": [0],
    "Aptitude": [0],
    "Communication": [0]
})

prediction = ann.predict(sample)

if prediction[0] == 1:
    print("\nPrediction : Yes (Placed)")
else:
    print("\nPrediction : No (Not Placed)")

```

Output:

*** Training Dataset

	CGPA	Aptitude	Communication	Placement
0	High	High	High	Yes
1	High	High	High	Yes
2	High	High	Medium	Yes
3	Low	Low	Low	No
4	Low	Low	Low	No
5	High	High	High	Yes
6	Low	Low	Medium	No
7	High	High	High	Yes
8	Low	Low	Low	No
9	High	High	High	Yes

Prediction : Yes (Placed)

5. Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python

Code:

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder

# Dataset

data = {
    "CGPA": ["High", "High", "High", "Low", "Low", "High", "Low", "High", "Low", "High"],
    "Aptitude":
["High", "High", "High", "Low", "Low", "High", "Low", "High", "Low", "High"],
    "Communication":
["High", "High", "Medium", "Low", "Low", "High", "Medium", "High", "Low", "High"],
    "Placement": ["Yes", "Yes", "Yes", "No", "No", "Yes", "No", "Yes", "No", "Yes"]
}

df = pd.DataFrame(data)

print("Training Dataset\n")
print(df)

encoder = LabelEncoder()

df["CGPA"] = encoder.fit_transform(df["CGPA"])
df["Aptitude"] = encoder.fit_transform(df["Aptitude"])
df["Communication"] = encoder.fit_transform(df["Communication"])
df["Placement"] = encoder.fit_transform(df["Placement"])

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X, y)

sample = pd.DataFrame({
    "CGPA": [0],
    "Aptitude": [0],
    "Communication": [0]
```

```
}}
```

```
prediction = knn.predict(sample)
```

```
if prediction[0] == 1:
```

```
    print("\nPrediction : Yes (Placed)")
```

```
else:
```

```
    print("\nPrediction : No (Not Placed)")
```

Output:

```
... Training Dataset
```

	CGPA	Aptitude	Communication	Placement
0	High	High	High	Yes
1	High	High	High	Yes
2	High	High	Medium	Yes
3	Low	Low	Low	No
4	Low	Low	Low	No
5	High	High	High	Yes
6	Low	Low	Medium	No
7	High	High	High	Yes
8	Low	Low	Low	No
9	High	High	High	Yes

```
Prediction : Yes (Placed)
```

6. Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

Code:

```
import pandas as pd
```

```
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
from sklearn.model_selection import train_test_split
```

```
# Dataset
```

```
data = {
```

```
    "CGPA": ["High","High","High","Low","Low","High","Low","High","Low","High"],
```

```
    "Aptitude":
```

```
["High","High","High","Low","Low","High","Low","High","Low","High"],
```

```

    "Communication":
["High","High","Medium","Low","Low","High","Medium","High","Low","High"],
    "Placement": ["Yes","Yes","Yes","No","No","Yes","No","Yes","No","Yes"]
}

```

```

df = pd.DataFrame(data)

```

```

print("Training Dataset\n")
print(df)

```

```

encoder = LabelEncoder()

```

```

df["CGPA"] = encoder.fit_transform(df["CGPA"])
df["Aptitude"] = encoder.fit_transform(df["Aptitude"])
df["Communication"] = encoder.fit_transform(df["Communication"])
df["Placement"] = encoder.fit_transform(df["Placement"])

```

```

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

```

```

model = GaussianNB()

```

```

model.fit(X_train, y_train)

```

```

y_pred = model.predict(X_test)

```

```

print("\nConfusion Matrix\n")
print(confusion_matrix(y_test, y_pred))

```

```

print("\nAccuracy =", round(accuracy_score(y_test, y_pred) * 100, 2), "%")

```

```

sample = pd.DataFrame({
    "CGPA": [0],
    "Aptitude": [0],
    "Communication": [0]
})

```

```

prediction = model.predict(sample)

if prediction[0] == 1:
    print("\nPrediction : Yes (Placed)")
else:
    print("\nPrediction : No (Not Placed)")

```

Output:

```

... Training Dataset

```

	CGPA	Aptitude	Communication	Placement
0	High	High	High	Yes
1	High	High	High	Yes
2	High	High	Medium	Yes
3	Low	Low	Low	No
4	Low	Low	Low	No
5	High	High	High	Yes
6	Low	Low	Medium	No
7	High	High	High	Yes
8	Low	Low	Low	No
9	High	High	High	Yes

```

Confusion Matrix

```

```

[[1 0]
 [0 2]]

```

```

Accuracy = 100.0 %

```

```

Prediction : Yes (Placed)

```